

Dispositivos Conectados

Sensores/atuadores I2C

2025/2026

Authors: Paulo C. Bartolomeu

1 Objetivos

- Configuração e utilização de um barramento I2C;
- Leitura e escrita num sensor de temperatura simples (TC74) através do barramento I2C;
- Interação com um multi-sensor (DHT20) através do barramento I2C.

2 Introdução

O uso de comunicações eficientes e de baixo consumo é crucial para suportar a expansão da Internet das Coisas (IoT). Neste sentido, a escolha do protocolo de comunicação interno é um fator determinante para o sucesso de qualquer projeto. Entre as várias opções, o barramento I2C (*Inter-Integrated Circuit*) é uma tecnologia de base muito relevante devido à sua simplicidade e economia.

Ao utilizar apenas duas linhas de comunicação – a linha de dados série (SDA) e a linha de relógio (SCL) – reduz drasticamente a complexidade da cablagem e do design das placas de circuito impresso (PCBs). Num mundo de dispositivos IoT cada vez mais miniaturizados e densos em componentes, esta característica minimalista traduz-se em redução de custos e espaço, fatores essenciais para a viabilidade de dispositivos como sensores inteligentes, *wearables* e atuadores remotos.

Mais do que uma economia de pinos, o barramento I2C permite que múltiplos dispositivos (desde sensores de temperatura e humidade a módulos de memória e conversores analógico-digitais) partilhem o mesmo par de fios, diferenciando-se através de endereços exclusivos. Esta natureza multi-mestre e multi-escravo promove uma arquitetura modular e escalável, permitindo aos *developers* integrar rapidamente novos sensores ou funcionalidades sem ter de redesenhar completamente o sistema.

Nesta aula iremos explorar a utilização do protocolo I2C para ler informação de temperatura e humidade de sensores, além de alterar as respetivas configurações de funcionamento.

3 Trabalho Prático

O trabalho prático desta aula desenvolve-se com base na [API de Referência para os periféricos do ESP32-C6](#) juntamente com o [ESP32-C6 Technical Reference Manual](#). Adicionalmente, **deverão ser consultadas as fichas técnicas dos sensores TC74 e DHT20** disponíveis na página do elearning na pasta `Recursos`.

3.1 I2CTOOL: Exploração de comunicações I2C com sensores simples

Nesta parte iremos utilizar a ferramenta `i2ctool` fornecida nos exemplos da *framework* ESP-IDF para demonstrar como desenvolver aplicações relacionadas com I2C. Este exemplo fornece uma ferramenta de linha de comando para configurar o barramento I2C, procurar dispositivos, ler, escrever, e examinar registros.

Comece por montar o circuito da Figura 1 na placa branca. Como pode observar, o objetivo é desenvolver um circuito com o sensor de temperatura TC74, incluindo duas resistências de *pull-up* de 4.7K Ohm.

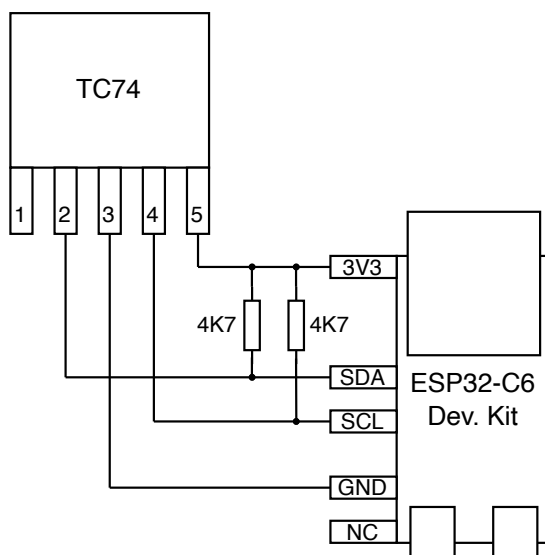


Figura 1: Circuito de ligação I2C ao sensor TC74

ATENÇÃO! Seja sempre prudente e **confirme as ligações na placa branca** antes de alimentar o ESP32.

Em seguida, através do *Wizard* da extensão ESP-IDF, crie um novo projeto tendo como base o exemplo `i2c` -> `i2ctool`. De seguida, compile o exemplo e descarregue-o para a placa.

Antes de ativar o monitor, troque a porta USB que está a utilizar de modo a ficar com uma interface série exclusiva para interagir com a ferramenta de linha de comando. Assim que alimentar a placa via a porta USB-UART, ative o Monitor e confirme que visualiza a seguinte informação:

```
I (186) main_task: Calling app_main()
```

```
=====
|           Steps to Use i2c-tools           |
|-----|-----|
| 1. Try 'help', check all supported commands |
| 2. Try 'i2cconfig' to configure your I2C bus |
| 3. Try 'i2cdetect' to scan devices on the bus |
| 4. Try 'i2cget' to get the content of specific register |
```

```

| 5. Try 'i2cset' to set the value of specific register |
| 6. Try 'i2cdump' to dump all the register (Experiment) |
|                                                         |
|=====|
Type 'help' to get the list of commands.
Use UP/DOWN arrows to navigate through command history.
Press TAB when typing command name to auto-complete.
I (366) main_task: Returned from app_main()
i2c-tools>

```

O *command prompt* `i2c-tools>` indica que a aplicação `i2c-tools` que está a ser executada na placa está pronta para receber um dos 6 comandos suportados: `help`, `i2cconfig`, `i2cdetect`, `i2cget`, `i2cset` e `i2cdump`.

QUESTÃO 1: Estude cada um dos comandos suportados e verifique qual é a funcionalidade associada. Em seguida, execute o comando `i2cdetect` e verifique o resultado. Deve obter uma tabela semelhante à apresentada em seguida. Como interpreta o resultado e porque é que ocorre?

```

i2c-tools> i2cdetect
    0  1  2  3  4  5  6  7  8  9  a  b  c  d  e  f
00: -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
10: -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
20: -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
30: -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
40: -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
50: -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
60: -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
70: -- -- -- -- -- -- -- -- -- -- -- -- -- -- --

```

O próximo passo é configurar através do comando `i2cconfig` quais são os portos que estão ligados ao relógio (sinal `SCL`) e aos dados série (sinal `SDA`), o que pode ser feito da seguinte forma:

```

i2c-tools> i2cconfig --sda 6 --scl 7

```

Execute novamente o comando `i2cdetect` e deve observar o seguinte resultado:

```

i2c-tools> i2cdetect
    0  1  2  3  4  5  6  7  8  9  a  b  c  d  e  f
00: -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
10: -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
20: -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
30: -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
40: -- -- -- -- -- -- -- 48 -- -- -- -- -- -- --
50: -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
60: -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
70: -- -- -- -- -- -- -- -- -- -- -- -- -- -- --

```

QUESTÃO 2: O que pode concluir deste resultado? Em que endereço está acessível o sensor de temperatura?

QUESTÃO 3: Analisando a ficha técnica do sensor e a operação da ferramenta `i2ctool`, execute o comando que lhe permita ler a temperatura ambiente instantânea. O resultado do comando deve conter a temperatura expressa em hexadecimal como ilustrado em seguida.

```

i2c-tools> <comando a usar>
0x19      <--- RESULTADO

```

QUESTÃO 4: Aqueça o sensor de temperatura soprando ar quente para o mesmo durante algum tempo. Repita o comando de leitura da temperatura e deverá observar pequenas alterações no valor lido.

QUESTÃO 5: Analise qual é o modo de funcionamento (`standby/normal`) do sensor TC74 lendo o respetivo `configuration register`. Interprete o resultado que deverá observar ser semelhante ao seguinte:

```
i2c-tools> <comando a usar>
0x40      <--- RESULTADO
```

QUESTÃO 6: Coloque o sensor a operar no modo de `standby`. Deverá observar o seguinte resultado:

```
i2c-tools> <comando a usar>
I (157146) cmd_i2ctools: Write OK
```

QUESTÃO 7: Aqueça novamente o sensor de temperatura soprando ar quente para o mesmo durante algum tempo. Repita o comando de leitura da temperatura. O que observa?

O processo descrito até aqui permite interagir com um sensor simples, lendo valores de registos e, quando suportado, escrever valores nos registos dos sensores.

3.2 I2CTOOL: Comunicações I2C com múltiplos sensores

ATENÇÃO! Sempre que pretender **fazer alterações elétricas** na placa, **desligue-a da alimentação**.

Monte na placa branca o circuito da Figura 2. Como documentado, o objetivo é adicionar o sensor de temperatura DHT20 ao barramento I2C, mantendo as duas resistências de *pull-up* de 4.7K Ohm existentes.

Ligue novamente a placa e execute o comando `i2cdetect`, devendo observar um resultado semelhante ao seguinte. Como pode observar, são detetados dois dispositivos I2C no barramento: o TC74 (`0x48`) e o DHT20 (`0x38`)

```
i2c-tools> i2cdetect
   0  1  2  3  4  5  6  7  8  9  a  b  c  d  e  f
00: -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
10: -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
20: -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
30: -- -- -- -- -- -- -- 38 -- -- -- -- -- -- --
40: -- -- -- -- -- -- -- 48 -- -- -- -- -- -- --
50: -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
60: -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
70: -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
i2c-tools>
```

TROUBLESHOOTING! Se fizer reset à placa necessitará novamente de configurar os pinos de SDA e SCL através do comando `i2cconfig`.

QUESTÃO 8: Analise a ficha técnica do sensor DHT. Consegue explicar como realizar uma leitura da temperatura e humidade deste sensor? Qual será a sequência de *bytes* a enviar para o sensor e como ler o resultado?

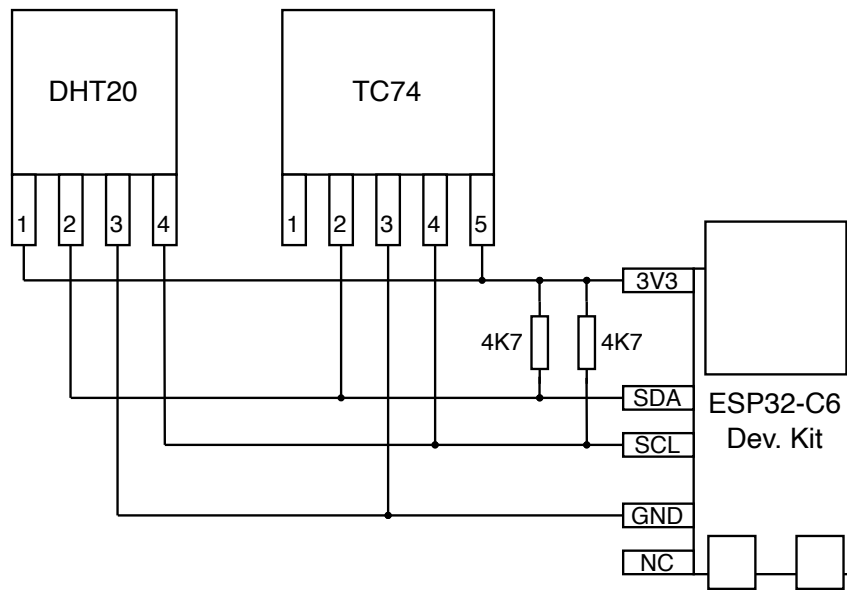


Figura 2: Circuito de ligação I2C aos sensores TC74 e DHT20

QUESTÃO 9: Tirando partido dos comandos `i2cset` e `i2cget`, faça uma leitura da temperatura e humidade. Converta o resultado de modo a obter as respetivas temperatura e humidade.

3.3 Integração de sensores I2C

Nesta secção pretende-se desenvolver um programa que force a leitura da temperatura do sensor TC74 a cada 2 segundos, imprimindo o resultado para o Monitor representado em graus Célsius. Em seguida, representa-se parte do código fornecido no ficheiro “tc74_test.c”.

```
#include ....

/* ---- I2C configuration ---- */
#define I2C_MASTER_SCL_IO      <TBD>
#define I2C_MASTER_SDA_IO      <TBD>
#define I2C_MASTER_NUM         I2C_NUM_0
#define I2C_MASTER_FREQ_HZ     <TBD>
#define I2C_MASTER_TX_BUF_DISABLE 0
#define I2C_MASTER_RX_BUF_DISABLE 0
#define I2C_MASTER_TIMEOUT_MS  1000

/* ---- TC74 specifics ---- */
#define TC74_ADDR               <TBD>
#define TC74_REG_TEMP           <TBD>
#define TC74_REG_CONFIG         <TBD>

...

void app_main(void)
{
    uint8_t temp_reg;
    uint8_t config_reg;
    i2c_master_bus_handle_t bus_handle;
```

```

i2c_master_dev_handle_t dev_handle;

i2c_master_init(&bus_handle, &dev_handle);
ESP_LOGI(TAG, "I2C initialized successfully");

// Read configuration register
ESP_ERROR_CHECK(tc74_register_read(dev_handle, TC74_REG_CONFIG, &config_reg, 1));
ESP_LOGI(TAG, "Initial config = 0x%02X", config_reg);

// Ensure sensor is in normal mode (bit7 = 0)
if (config_reg & 0x80) {
    ESP_LOGW(TAG, "Sensor in standby, switching to normal mode...");
    config_reg &= 0x7F;
    ESP_ERROR_CHECK(tc74_register_write_byte(dev_handle, TC74_REG_CONFIG, config_reg));
    vTaskDelay(pdMS_TO_TICKS(200));
}

while (1) {
    ESP_ERROR_CHECK(tc74_register_read(dev_handle, TC74_REG_TEMP, &temp_reg, 1));

    int8_t temp_signed = (int8_t)temp_reg; // Temperatura in C
    ESP_LOGI(TAG, "Temperatura: %d C", temp_signed);

    vTaskDelay(pdMS_TO_TICKS(2000));
}

// Normally we never reach this, but here for completeness:
ESP_ERROR_CHECK(i2c_master_bus_rm_device(dev_handle));
ESP_ERROR_CHECK(i2c_del_master_bus(bus_handle));
ESP_LOGI(TAG, "I2C de-initialized successfully");
}

```

QUESTÃO 10: Estude a [API de referência do I2C](#), particularmente a [componente de alocação e liberação de recursos](#). Verifique como é lido um registo.

QUESTÃO 11: Estude e complete o código fornecido no ficheiro “tc74_test.c” (modificando os valores TBD) de modo a compatibilizá-lo com o sensor TC74. Teste o código e aqueça o sensor com ar quente, confirmando que o resultado é semelhante ao seguinte:

```

I (243) TC74: I2C initialized successfully
I (253) TC74: Initial config = 0x40
I (253) TC74: Temperatura: 25 C
I (2253) TC74: Temperatura: 25 C
I (4253) TC74: Temperatura: 25 C
I (6253) TC74: Temperatura: 25 C
I (8253) TC74: Temperatura: 26 C
I (10253) TC74: Temperatura: 26 C

```

QUESTÃO 12: Partindo do código fornecido, realize as alterações necessárias a compatibilizá-lo com a realização de leituras do sensor DHT20. Teste o código e aqueça o sensor, confirmando que o resultado é semelhante ao seguinte:

```

I (8453) DHT20: Temperatura: 22.60 C, Humidade: 69.61 %
I (10493) DHT20: Temperatura: 22.61 C, Humidade: 69.64 %
I (12533) DHT20: Temperatura: 22.76 C, Humidade: 71.17 %
I (14573) DHT20: Temperatura: 23.49 C, Humidade: 74.63 %
I (16613) DHT20: Temperatura: 23.94 C, Humidade: 77.58 %
I (18653) DHT20: Temperatura: 24.24 C, Humidade: 80.10 %
I (20693) DHT20: Temperatura: 24.37 C, Humidade: 82.08 %

```

4 Acknowledgements

Originally authored by [Paulo C. Bartolomeu](#)